

Table 1 Tools for designing DNA sequence libraries and for related sequence-design problems. Members of the final two rows are named in the table and cited individually. Tools were examined in August 2026.

Tool	Purpose	Key features	Output	Availability	Ref.
MPRA Design Tools	Design MPRA libraries and estimate statistical power	Barcode assignment with error-correcting sets; power analysis over effect size and sequencing depth	Barcoded constructs and statistical power estimates	R package (GitHub)	[1]
MPRAnator	Design MPRA libraries of motif arrangements and sequence variants	Combinatorial placement of motifs; randomly sampled and combinatorial substitution sets; scrambled and reverse-complement controls	Barcoded library of sequences (FASTA)	Web service	[2]
Mutation Maker	Design mutagenic primers for laboratory construction of variant libraries	Three workflows: site-scanning saturation mutagenesis, multi-site directed mutagenesis, and PCR-based gene synthesis; degenerate codons and primer mixing ratios	Mutagenic primer pairs, or gene fragments with mixing ratios	Web application (Docker); REST API	[3]
Oligopool Calculator	Add barcodes, primers, and spacers to a supplied variant set, and analyze the sequencing readout	Constraint-aware barcodes, primers, and spacers; degenerate compression of a supplied variant set; off-target screening against a host genome	Synthesis-ready constructs; variant counts from sequencing reads	Python package (PyPI); command line	[4]
PoolParty	Design libraries combining multiple variant types, for laboratory assays or for probing genomic AI models	Libraries specified as a directed acyclic graph of operations, freely mixing mutagenesis schemes; inputs from sequences, FASTA, degenerate templates, or motifs; sequences generated only on request	Library of sequences, each paired with a record of how it was constructed	Python package (PyPI)	This work
	Generate perturbed sequence sets to probe cis-regulatory logic in deep learning models	Saturation mutagenesis of input sequences; insertion and removal of motifs across sets of sequences	Perturbed sequences, model predictions, or attribution scores	Python package (PyPI)	[5]
VaLiAnT	Design and annotate libraries for saturation genome editing and cDNA deep mutational scanning	Saturation mutagenesis from genomic coordinates; several mutation types per target region; transcript-aware, including codons split across exons	Per-region libraries with metadata tables and VCF carrying variant consequence annotation	Command-line tool (source, Docker)	[6]
General-purpose toolkits (<i>Biopython</i> , <i>pydna</i> , <i>SeqPro</i>)	Manipulate sequences, simulate cloning, prepare arrays for modeling	Parsing, transformation, and file-format support	Transformed sequences	Python packages (PyPI)	[7–9]
Sequence optimization tools (<i>CodonGenie</i> , <i>DNA Chisel</i> , <i>ledidi</i>)	Optimize individual sequences for codon usage, synthesis constraints, or a model’s objective	Degenerate codons covering a target residue set; hard constraints and scored objectives combined in one problem; gradient-based editing guided by a predictive model	An optimized or edited sequence, or a degenerate codon	Python packages (PyPI); web services	[10–12]

Table 2 Capabilities of tools for designing DNA sequence libraries. ● denotes a capability the tool provides; ◐ one provided with restrictions; ○ one the tool does not provide. Capabilities are counted only where the tool provides a dedicated operation, parameter, or mode. Tools were assessed in August 2026.

	DNA Chisel	MPRA Design Tools	MPRAnator	Mutation Maker	Oligopool Calculator	PoolParty	tangermeme	ValiAnT
<i>Library Specification</i>								
Library object	○	◐	○	◐	◐	●	◐	○
Composable operations	◐	○	◐	○	●	●	◐	○
Mixed variant types in one library	○	●	●	◐	○	●	○	●
<i>Variant Generation</i>								
Saturation mutagenesis	○	○	○	○	◐	●	●	●
Randomly sampled variants	●	◐	●	○	○	●	○	○
Pairwise and higher-order variants	●	○	●	●	◐	●	◐	○
Model-guided variants	●	○	○	○	○	◐	●	○
<i>Variant Types</i>								
Codon / amino-acid substitutions	●	○	○	◐	○	●	○	●
Insertions and deletions	○	●	◐	○	○	●	●	●
Combinatorial multi-motif placement	○	○	●	○	○	●	●	○
Recombination	○	○	○	○	○	●	○	○
Shuffling	○	○	◐	○	○	●	●	○
<i>Physical Construction</i>								
Synthesis-constraint checking	●	◐	◐	○	●	◐	○	◐
Constraint-based optimization	●	◐	◐	●	●	○	○	○
Primer design	◐	○	○	●	●	○	○	○
<i>Metadata and Inspection</i>								
Per-sequence design cards	●	●	◐	●	●	●	○	●
Automatic naming	○	◐	●	●	◐	●	◐	●
Sequence styling	○	○	◐	○	○	●	○	○
<i>Genomic Integration</i>								
Genome coordinates	○	◐	●	○	○	◐	◐	●
Transcript / annotation aware	○	○	○	○	○	○	○	●

Table 3 Runtime and memory for the three worked examples. Each library was generated at its published size on a single core. Times are means $\pm$ SD over 5 replicate runs; peak memory is maximum resident set size, including a ~ 106 MB baseline for the Python interpreter, NumPy, and pandas. DAG construction does not depend on library size and is reported separately. The SpliceAI library comprises two matched 200,000-sequence pools (cryptic GT and disrupted GA) generated separately; values are for one pool, and the full library takes twice as long.

Example	Library size	Generation time (s)	Throughput (seq s ⁻¹)	Peak memory (MB)	DAG construction (s)
GB1 deep mutational scan	547,230	132.24 $\pm$ 5.76	4,138	487	0.22
MPRA regulatory grammar	24,000	9.22 $\pm$ 0.22	2,602	127	0.06
SpliceAI surrogate	200,000	11.93 $\pm$ 0.27	16,758	208	0.05

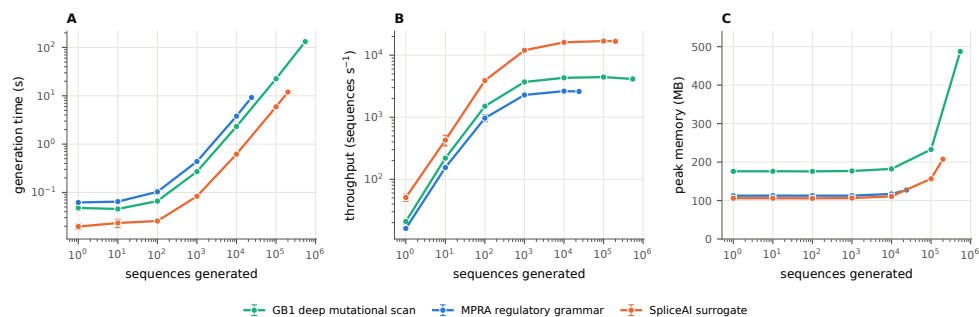


Fig. 1 Library generation scales linearly in time and memory. (A) Wall-clock generation time, (B) throughput, and (C) peak resident memory against the number of sequences generated, for the three worked examples. Points are means of 5 replicate runs; error bars show ± 1 SD and are smaller than the plotting symbols at most points. Below $\sim 10^3$ sequences a fixed setup cost dominates, giving the plateau in (A) and the rise in (B); above 10^4 sequences throughput is constant, so generation time is linear in library size. Peak memory increases linearly, by 0.52–0.66 kB per sequence above a ~ 106 MB baseline.

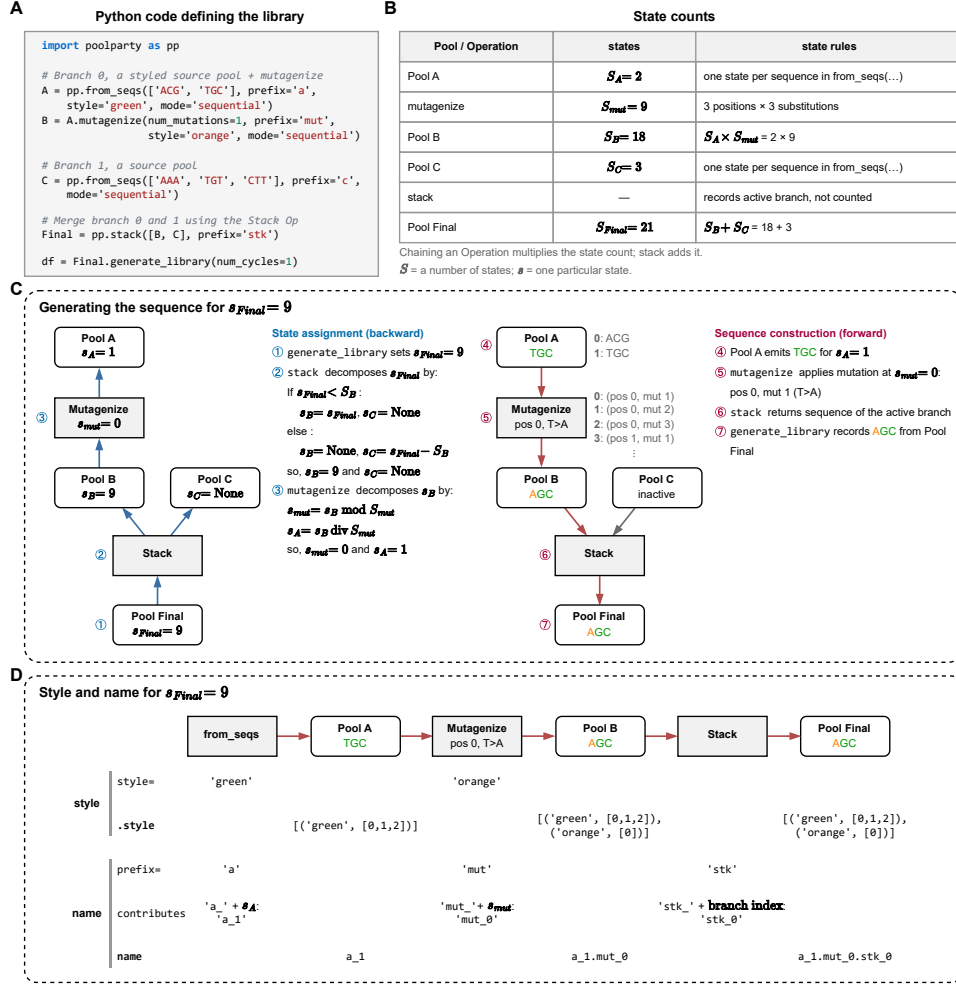


Fig. 2 A worked example of sequence generation. (A) Python code defining the library. (B) Number of states of each Pool and Operation. (C) Generating the sequence for $s_{Final} = 9$. Left: state assignment (blue). Right: sequence construction (magenta). Lists beside Pool A and **mutagenize** show which sequence or mutation each state selects. Inactive branches are shown in grey. (D) Style and name for the same sequence. Sequences are carried through the DAG as **Seq** objects, which hold the sequence string together with its **.style**. The **style** and **prefix** arguments are supplied to Operations; the resulting **.style** and name are held by Pools. Sequence characters are colored by their **.style** entries; where entries overlap, the last one applies.

References

- [1] Ghazi AR, Chen ES, Henke DM, Madan N, Edelstein LC, Shaw CA. Design tools for MPRA experiments. *Bioinformatics*. 2018;34(15):2682–2683. <https://doi.org/10.1093/bioinformatics/bty150>.
- [2] Georgakopoulos-Soares I, Jain N, Gray JM, Hemberg M. MPRAinator: a web-based tool for the design of massively parallel reporter assay experiments. *Bioinformatics*. 2017;33(1):137–138. <https://doi.org/10.1093/bioinformatics/btw584>.
- [3] Hiraga K, Mejzlik P, Marcisin M, Vostrosablin N, Gromek A, Arnold J, et al. Mutation Maker, An Open Source Oligo Design Platform for Protein Engineering. *ACS Synth Biol*. 2021;10(2):357–370. <https://doi.org/10.1021/acssynbio.0c00542>.
- [4] Hossain A, Cetnar DP, LaFleur TL, McLellan JR, Salis HM. Automated Design of Oligopools and Rapid Analysis of Massively Parallel Barcoded Measurements. *ACS Synth Biol*. 2024;13(12):4218–4232. <https://doi.org/10.1021/acssynbio.4c00661>.
- [5] Schreiber J. tangermeme: A toolkit for understanding cis-regulatory logic using deep learning models. *bioRxiv*. 2025;p. 2025.08.08.669296. <https://doi.org/10.1101/2025.08.08.669296>.
- [6] Barbon L, Offord V, Radford EJ, Butler AP, Gerety SS, Adams DJ, et al. Variant Library Annotation Tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments. *Bioinformatics*. 2022;38(4):892–899. <https://doi.org/10.1093/bioinformatics/btab776>.

- [7] Cock PJA, Antao T, Chang JT, Chapman BA, Cox CJ, Dalke A, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*. 2009;25(11):1422–1423. <https://doi.org/10.1093/bioinformatics/btp163>.
- [8] Pereira F, Azevedo F, Carvalho Â, Ribeiro GF, Budde MW, Johansson B. Pydna: a simulation and documentation tool for DNA assembly strategies using python. *BMC Bioinformatics*. 2015;16(1):142. <https://doi.org/10.1186/s12859-015-0544-x>.
- [9] Klie A, Laub D. SeqPro (Sequence processing toolkit); 2023. Software. Available from: <https://github.com/ML4GLand/SeqPro>.
- [10] Swainston N, Currin A, Green L, Breitling R, Day PJ, Kell DB. CodonGenie: optimised ambiguous codon design tools. *PeerJ Comput Sci*. 2017;3:e120. <https://doi.org/10.7717/peerj-cs.120>.
- [11] Zulkower V, Rosser S. DNA Chisel, a versatile sequence optimizer. *Bioinformatics*. 2020;36(16):4508–4509. <https://doi.org/10.1093/bioinformatics/btaa558>.
- [12] Schreiber J, Lorbeer FK, Heinzl M, Lu YY, Stark A, Noble WS. Programmatic design and editing of cis-regulatory elements. *bioRxiv*. 2025;p. 2025.04.22.650035. <https://doi.org/10.1101/2025.04.22.650035>.